

CWM WORK JOURNAL ASSIGNMENT 2

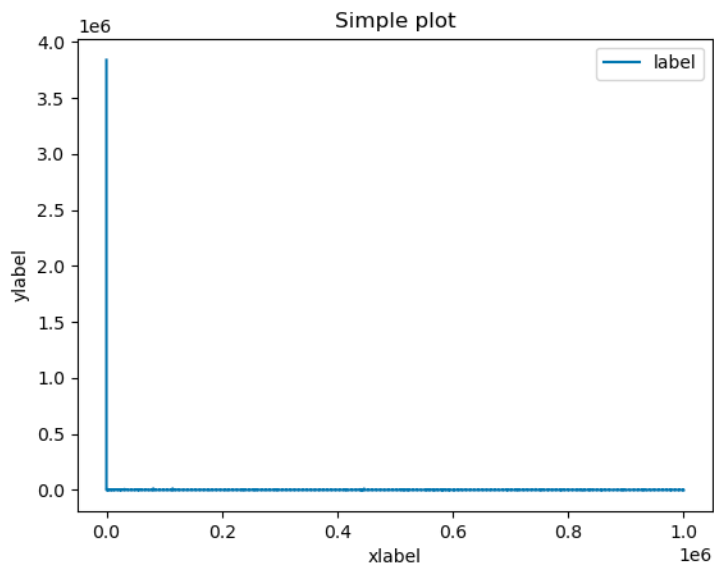
Q1

The test results of the time gap between 2 consecutive reads in CPU.

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246697646117146 ns
CPU time diff: 15033 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246703413124711 ns
CPU time diff: 14944 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246704436995252 ns
CPU time diff: 21256 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246704924735391 ns
CPU time diff: 15536 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246705332924843 ns
CPU time diff: 14082 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246705700807135 ns
CPU time diff: 15140 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246706034509738 ns
CPU time diff: 20008 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246706365040722 ns
CPU time diff: 13223 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246706572983735 ns
CPU time diff: 15693 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdttime.py
CPU time counter: 246708277053786 ns
CPU time diff: 14444 ns
```

Q2:

The results show a pretty long time difference at the start and quickly enters a steady state with a relatively smaller range of variation. The large gap in the beginning is caused by the initialization processes when starting the program.



Q3: Run using C program

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.200 GHz
Minimum consecutive TSC diff: 192 cycles
Approximate time: 159.99 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.135 GHz
Minimum consecutive TSC diff: 190 cycles
Approximate time: 167.42 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.100 GHz
Minimum consecutive TSC diff: 210 cycles
Approximate time: 190.91 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.200 GHz
Minimum consecutive TSC diff: 194 cycles
Approximate time: 161.66 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.112 GHz
Minimum consecutive TSC diff: 194 cycles
Approximate time: 174.40 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.200 GHz
Minimum consecutive TSC diff: 194 cycles
Approximate time: 161.66 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.300 GHz
Minimum consecutive TSC diff: 180 cycles
Approximate time: 138.46 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.300 GHz
Minimum consecutive TSC diff: 172 cycles
Approximate time: 132.31 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.201 GHz
Minimum consecutive TSC diff: 186 cycles
Approximate time: 154.93 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 1.228 GHz
Minimum consecutive TSC diff: 192 cycles
Approximate time: 156.36 ns
```

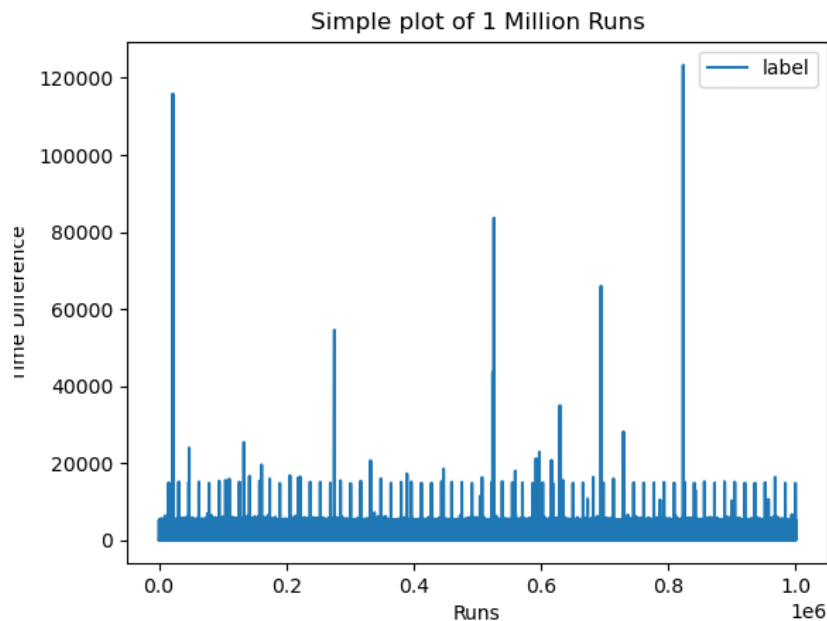
Q4: Run Using Power Mode

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 18.33 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 18.33 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
```

Using power mode, the CPU gap time is much shorter than the results run using balanced mode.

Q5: C code run 1 million times

For C program, the initial huge time difference disappears, and the time difference goes into a steady small range faster than the python program.



Q6: Taskset Pinned Experiment

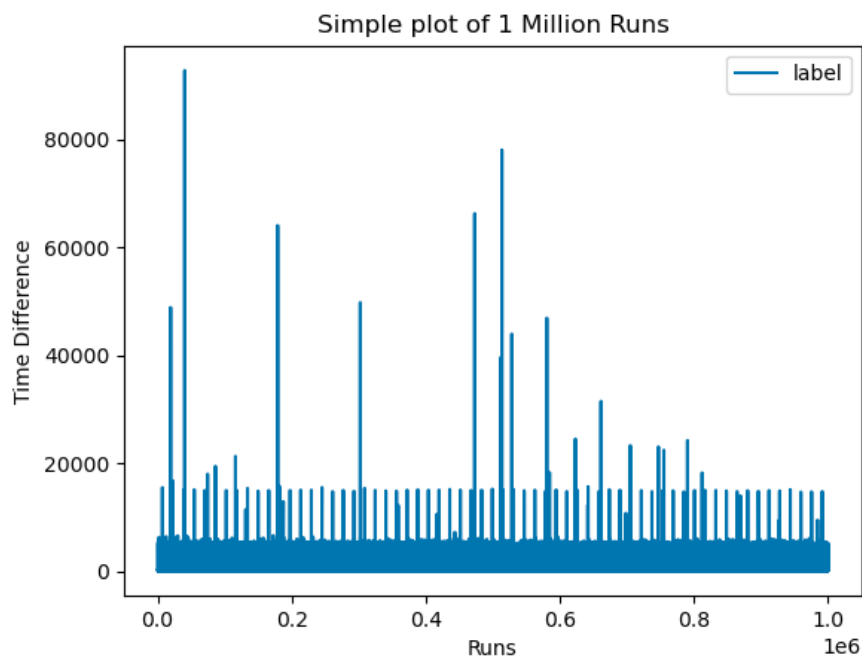
```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 1 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
```

For the pinned experiment, I pin the task to core 1. The results are identical to the unpinned experiment. And I also tried pinning the task to other cores, which also showed identical results. These results implied that the CPU cores are synchronized. By observing the average value between pinned and unpinned results, I found out that the average values are also similar. However, the peaks in unpinned experiments are much higher than unpinned ones. This means that the 4 cores are working with similar loadings, but for unpinned experiments, interruptions in any cores might influence the program, leading to higher peaks.

```

ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 2 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 2 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 3 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 4 ./rdtime 1000000
taskset: failed to set pid 36735's affinity: Invalid argument
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 5 ./rdtime 1000000
taskset: failed to set pid 36737's affinity: Invalid argument
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 3 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 0 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns

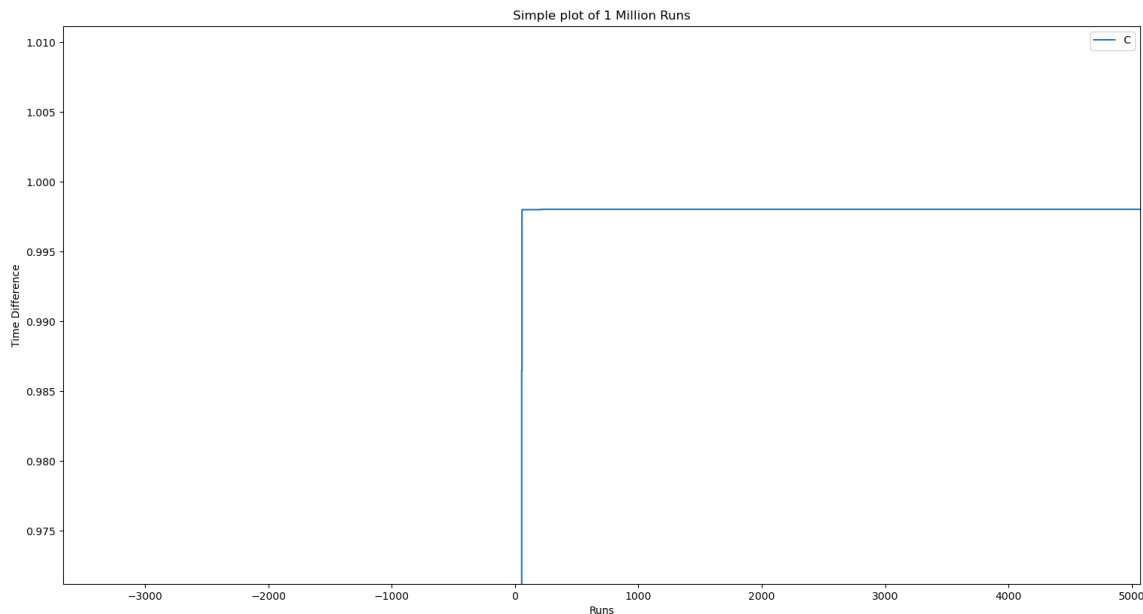
```



Q7: Statistics of C Code

The minimum is 44 cycles shown in the code. The average and median is about 52 by reading the output. Also, by observation, the 90 and 99th percentiles are located around 58 to 62.

However, the maximum is located at 160000, much larger than other statistical values. This is due to the heavy tail distribution properties.



Q8:

Python and C experiments have significant differences in their results. For python codes, the average time cycles difference is higher than that of C codes. Besides, Python codes always have a huge time difference at the start of the program, then entering a steady range. However, the C code does not have this transient behavior. This is because running a python code requires the CPU to set up python compilers and environment, which takes a long time to do that. However, these procedures are not required for C codes.

Q9:

The maximum time gaps occurred because of interruptions of the program due to other instructions running in the core. This waiting of other instructions leads to the huge time gaps occurring in the results. However, as these cases are scarce compared to the program, the median is significantly smaller.

Q10:

I pinned Google 10 times, and this is my result.

```

ubuntu@ubuntu:~/CWM-FDI/scripts$ ping 8.8.8.8 -c 10 -i 0.2
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=3.29 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=3.43 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=3.35 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=3.37 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=3.28 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=113 time=3.27 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=113 time=3.26 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=113 time=3.42 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=113 time=3.44 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=113 time=3.26 ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1807ms
rtt min/avg/max/mdev = 3.260/3.338/3.443/0.070 ms

```

Q11:

Ping

www.ox.ac.uk

```

ubuntu@ubuntu:~/CWM-FDI/scripts$ ping www.ox.ac.uk -c 10 -i 0.2
PING www.ox.ac.uk.cdn.cloudflare.net (172.66.169.161) 56(84) bytes of data.
64 bytes from 172.66.169.161: icmp_seq=1 ttl=52 time=8.64 ms
64 bytes from 172.66.169.161: icmp_seq=2 ttl=52 time=8.63 ms
64 bytes from 172.66.169.161: icmp_seq=3 ttl=52 time=8.73 ms
64 bytes from 172.66.169.161: icmp_seq=4 ttl=52 time=8.80 ms
64 bytes from 172.66.169.161: icmp_seq=5 ttl=52 time=8.77 ms
64 bytes from 172.66.169.161: icmp_seq=6 ttl=52 time=8.73 ms
64 bytes from 172.66.169.161: icmp_seq=7 ttl=52 time=8.64 ms
64 bytes from 172.66.169.161: icmp_seq=8 ttl=52 time=8.86 ms
64 bytes from 172.66.169.161: icmp_seq=9 ttl=52 time=8.63 ms
64 bytes from 172.66.169.161: icmp_seq=10 ttl=52 time=8.84 ms

--- www.ox.ac.uk.cdn.cloudflare.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1809ms
rtt min/avg/max/mdev = 8.631/8.726/8.856/0.084 ms

```

Ping

www.pku.edu.cn

```

ubuntu@ubuntu:~/CWM-FDI/scripts$ ping www.pku.edu.cn -c 10 -i 0.2
PING www.lb.pku.edu.cn (162.105.131.160) 56(84) bytes of data.

--- www.lb.pku.edu.cn ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 1870ms

```

Not receiving anything


```
ubuntu@ubuntu:~/CWM-FDI/scripts$ ping ethz.ch -c 10 -i 0.2
PING ethz.ch (129.132.19.216) 56(84) bytes of data.
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=1 ttl=49 time=21.8 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=2 ttl=49 time=22.0 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=3 ttl=49 time=22.0 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=4 ttl=49 time=22.0 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=5 ttl=49 time=21.8 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=6 ttl=49 time=21.7 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=7 ttl=49 time=21.9 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=8 ttl=49 time=21.7 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=9 ttl=49 time=22.0 ms
64 bytes from cms-publish.ethz.ch (129.132.19.216): icmp_seq=10 ttl=49 time=21.9 ms

--- ethz.ch ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1808ms
rtt min/avg/max/mdev = 21.693/21.876/21.965/0.095 ms
```

Pinning farther universities will lead to a longer time lag. If the time lag is too long, like the example I pinned a university in China, the computer did not receive any data pack back.

Q12: Pinning the computer itself

```
ubuntu@ubuntu:~/CWM-FDI/scripts$ ping 127.0.0.1 -c 10 -i 0.2
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.015 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.017 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.016 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.019 ms

--- 127.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1870ms
rtt min/avg/max/mdev = 0.015/0.017/0.019/0.001 ms
```

Q13: Pinning the computer itself 100 times 0.01 interval

```
--- 127.0.0.1 ping statistics ---
100 packets transmitted, 100 received, 0% packet loss, time 1093ms
rtt min/avg/max/mdev = 0.007/0.017/0.044/0.004 ms
```

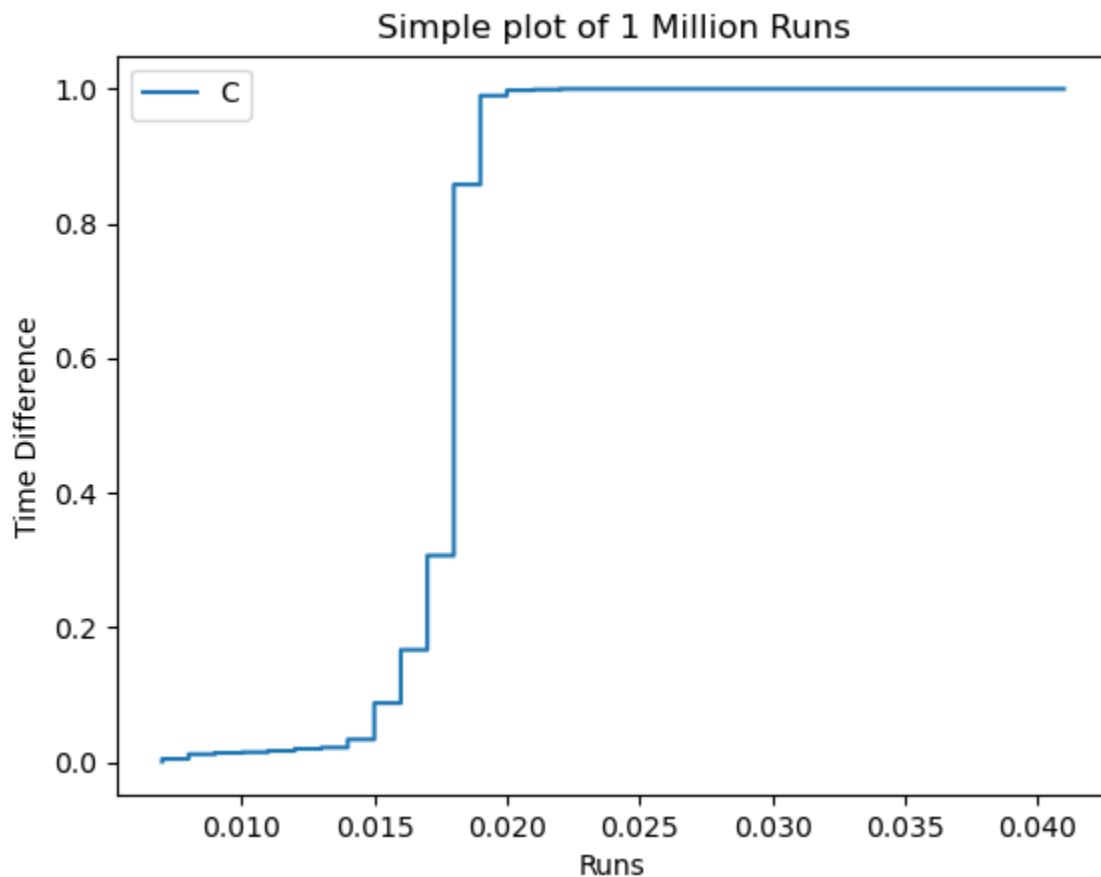
The average and minimum values are close, but the maximum is 2 times larger than the average value. Overall, both the minimum and maximum values occur seldom in the results.

Q14: Pinning 10000 times


```
--- 127.0.0.1 ping statistics ---
10000 packets transmitted, 10000 received, 0% packet loss, time 110316ms
rtt min/avg/max/mdev = 0.007/0.016/0.061/0.002 ms
```

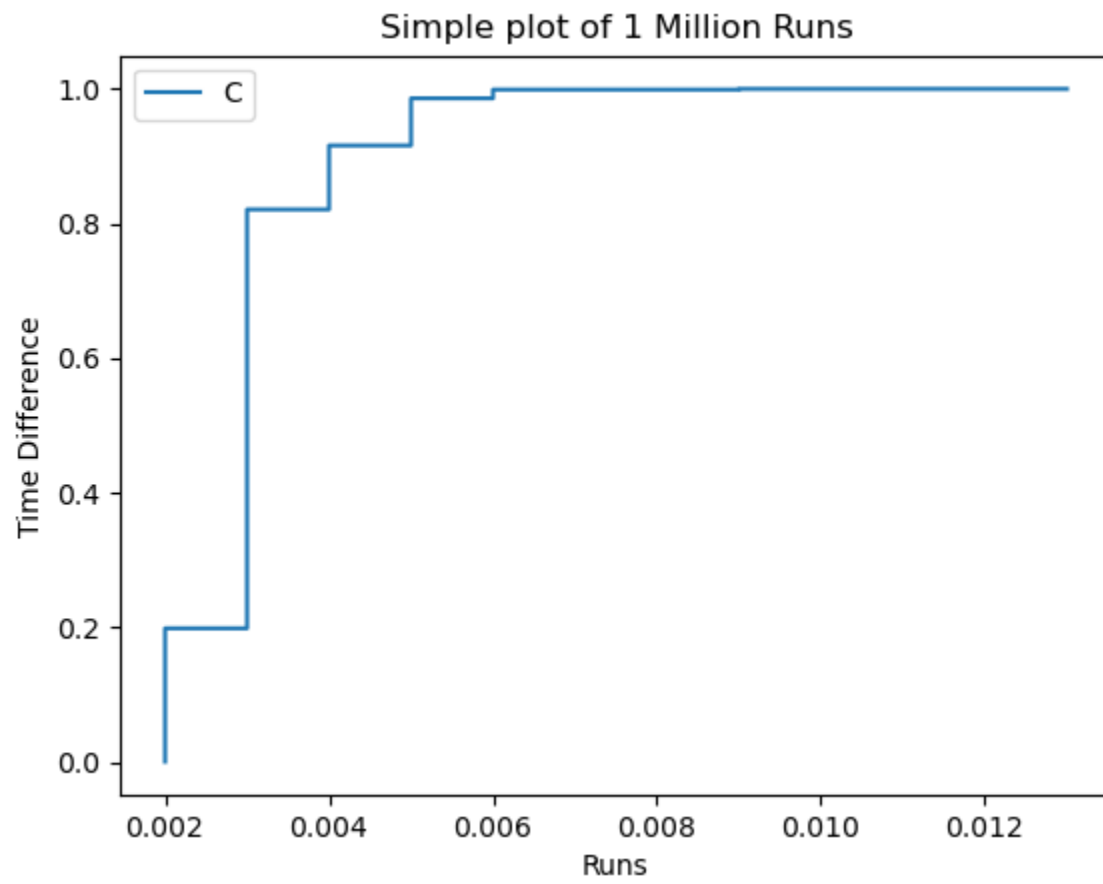
Q15: Ping tests

Test result of 0.01 interval.

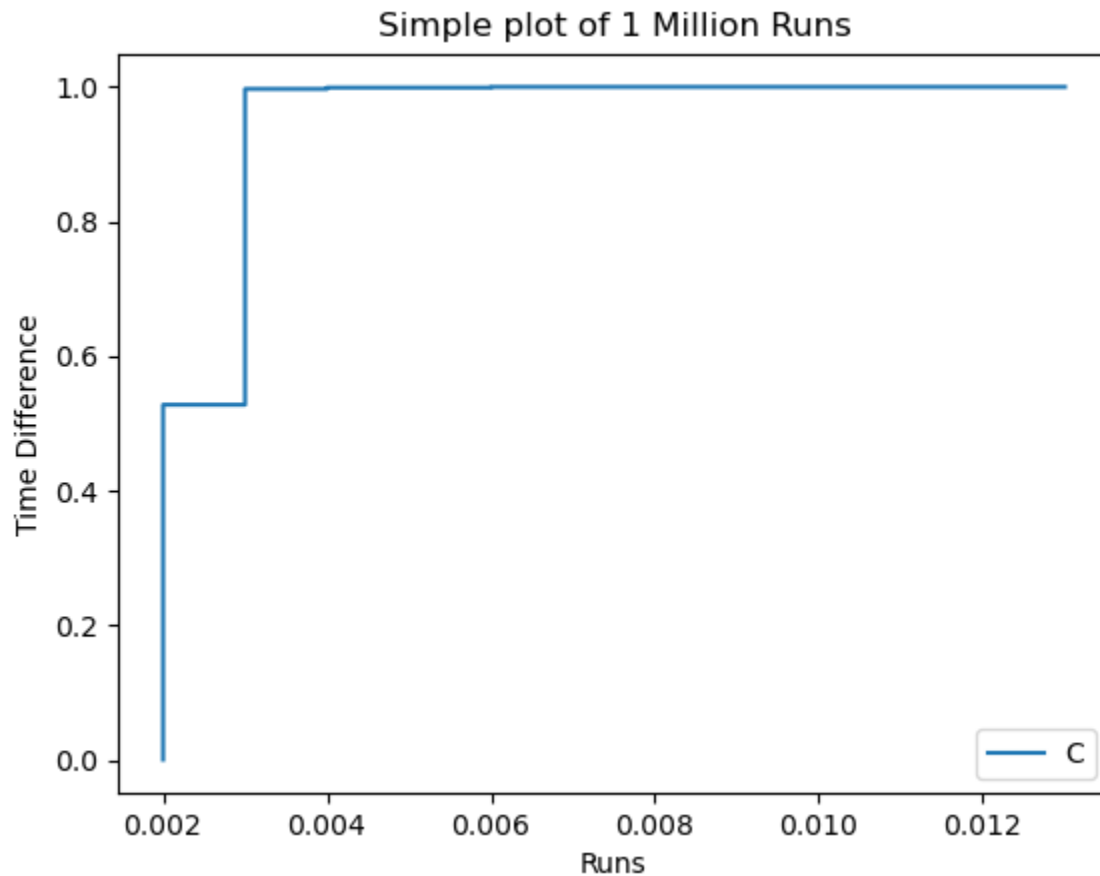


```
--- 127.0.0.1 ping statistics ---
909 packets transmitted, 909 received, 0% packet loss, time 10020ms
rtt min/avg/max/mdev = 0.007/0.016/0.047/0.002 ms
```

Ping test results of 0.001 interval



Test results of 0.0001 interval



Q16:

Shorter ping results will lead to shorter round-trip time. When the computer is pinning faster, the CPU will operate faster to process with instructions. However, this will come to a limit as the system clock has a maximum operation frequency. When approaching this frequency, the queries might build up in the CPU, causing the RTT to increase a little bit.

Q17:

An very effective way to achieve this is to run the computer in a power-saving mode. This will limit the computing power of the CPU largely, which will lead to a longer RTT as the CPU is processing instructions slower.

Q18:

The main idea is to remove everything apart from calculating the TSC. Therefore, there's no need to record every timestamp in an array and take the difference. Only consecutive ones are needed.

Also, there is no need to write the results into a file.
Q19:

From the previous code of `rdtime`, we know about the cycles it takes for one TSC reading. Therefore, we just need to use the average RTT time for a single ping request to divide the average cycles for one TSC reading to get the number of TSC readings per request.

For example, from previous test results, the minimum cycle for one TSC reading is 44, and the average value is around 52.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ ping 8.8.8.8 -c 10
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=3.18 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=3.24 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=3.44 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=3.17 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=3.28 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=113 time=3.17 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=113 time=3.38 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=113 time=3.43 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=113 time=3.23 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=113 time=3.19 ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9013ms
rtt min/avg/max/mdev = 3.166/3.270/3.437/0.101 ms
```

I ping Google 10 times and derive an average lag of 3.27ms. Therefore, the number of TSC per request can be calculated as follows:

$$3.27 \cdot 0.001 \cdot 3.6 \cdot \frac{10^9}{52} = 226384.6154$$

Q20 :

For this question, I asked Chatgpt about functions and methods of recording time in Python codes.

How to record and count times in python?

The two most common ways are `time.time()` and `time.perf_counter()`. For performance measurements, use `time.perf_counter()`.

Measure elapsed time

```
Python Run

import time

start = time.perf_counter()

# code to measure
for i in range(1000000):
    pass

end = time.perf_counter()

elapsed = end - start
print(f"Elapsed time: {elapsed:.6f} seconds")
```

📄 👍 💬 ⬆️ ↺ ...

what does perf_counter() do?

📄 ⬆️ ✎

`time.perf_counter()` returns a **high-resolution timer value** intended for measuring elapsed time.

Think of it as a stopwatch:

```
Python Run

import time

start = time.perf_counter()

# code to measure

end = time.perf_counter()

elapsed = end - start
```

The actual number returned is not meaningful by itself:

Q21:

Cache misses are not the only story here. The most important criterion is the cache miss rate, which also depends on the total references made by the program. When a program has more cache misses with more references, its cache missed rate might be lower, saying that the program is efficient.

In this question, I asked GPT about getting the cache miss rate using sudo perf command in the terminal.

in sudo perf instructions, may I know the cache-miss rate for the code I executed?

Yes. If you're using Linux `perf`, you can measure cache misses and calculate a cache miss rate.

Quick command

```
<> Bash
sudo perf stat -e cache-references,cache-misses ./your_program
```

Example output:

```
1,234,567    cache-references
45,678       cache-misses

3.70%        of all cache refs
```

The miss rate is:

$$\text{cache miss rate} = \frac{\text{cache-misses}}{\text{cache-references}} \times 100\%$$

In this example:

$$\frac{45678}{1234567} \times 100\% \approx 3.70\%$$

With this instruction, I tested all three fast algorithms and the slow one.

Performance of matmul_slow

```
Performance counter stats for 'python3 matmul_slow.py' (5 runs):

 807722682    cycles                               ( +- 0.45% )
2295526521    instructions          #    2.84  insn per cycle     ( +- 0.03% )
 713149       cache-misses          #    5.97% of all cache refs ( +- 7.60% )
11949018      cache-references          ( +- 2.63% )

0.22614 +- 0.00111 seconds time elapsed ( +- 0.49% )
```

Performance of matmul_fast3

```
Performance counter stats for 'python3 matmul_fast.py' (5 runs):

639553344     cycles                               ( +- 0.24% )
1837185512    instructions          #    2.87  insn per cycle     ( +- 0.04% )
 629324       cache-misses          #    5.56% of all cache refs ( +- 4.33% )
11326436      cache-references          ( +- 0.26% )

0.179341 +- 0.000497 seconds time elapsed ( +- 0.28% )
```

Performance of matmul_fast2

```

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

    756667541      cycles                               ( +-  0.09% )
    2273309992    instructions                #    3.00  insn per cycle  ( +-  0.04% )
      812697      cache-misses                #    6.40% of all cache refs ( +-  2.67% )
    12700626      cache-references                ( +-  0.24% )

    0.211988 +- 0.000438 seconds time elapsed ( +-  0.21% )

```

Performance of matmul_fast1

```

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

    739854120      cycles                               ( +-  0.55% )
    2060023288    instructions                #    2.78  insn per cycle  ( +-  0.05% )
      806021      cache-misses                #    6.95% of all cache refs ( +-  5.11% )
    11602040      cache-references                ( +-  1.73% )

    0.20737 +- 0.00129 seconds time elapsed ( +-  0.62% )

```

The results show that only the third algorithm has a smaller cache miss rate than the original slow algorithm. From this aspect, the results reveal that the operation speed of a program depends not only on cache missed but also on cycles run and instructions executed. However, to find the optimal one, must utilize the cache best and reduce cycle numbers the most, in this case matmul_fast 3.

The underlying reason might be that transposing B allows the code to access B's rows consecutively, which utilizes spatial locality well. In other words, this will lead to a lower cache miss rate and less instructions executed in reading missed values when doing multiplication.

Q22:

For future coding exercises, I come up with three worth memorized practices that I need to bring with me.

1. Consider temporal and spatial locality all the time. Maximum utilization of locality will largely reduce the run time and CPU load, achieving a more efficient and effective algorithm.
2. Reduce the number of loops in your algorithm. Avoid using many nested loops, which will generate a huge computing burden for your CPU. Break your nested loops to more efficient codes.
3. Use annotation cleverly to include changes that you need in some cases.